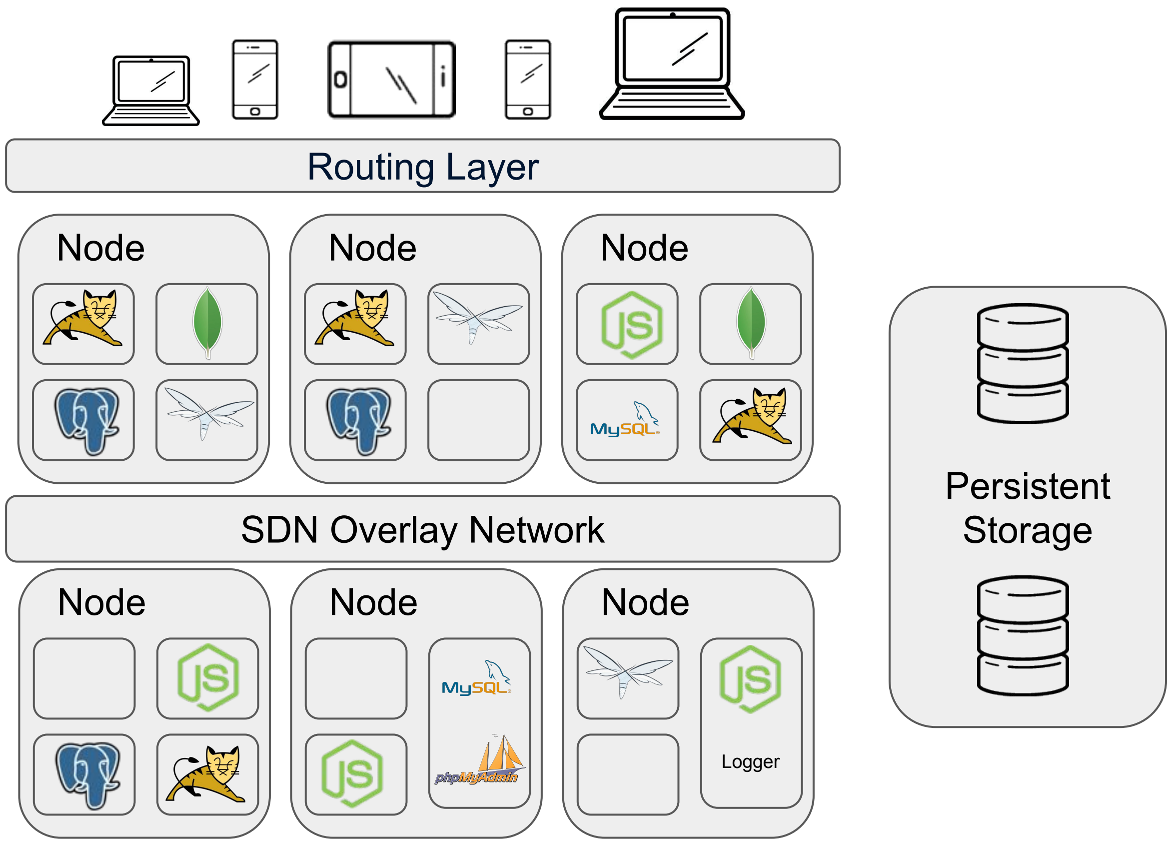
Kubernetes

Container Orchestrators

Karim AMMOUS & Fabrice Jammes 09/2019

Why ?

- Because real applications don’t run in a single container
- Real production applications can’t be deployed on a single host



What else?

- Bring multiple hosts together and make them part of a cluster
- Schedule containers to run on different hosts
- Help containers running on one host reach out to containers running on other hosts in the cluster
- Bind containers and storage
- Bind containers of similar type to a higher-level construct, like services, so we don't have to deal with individual containers
- Keep resource usage in-check, and optimize it when necessary
- Allow secure access to applications running inside containers.

Layers and providers

	Layers	Providers
PaaS	Container Application Platform	CloudFoundary, Heroku, Openshift
CaaS	Container Orchestration	Kubernetes, Marathon, Swarm
	Containerization	Docker, Containerd, Rocket (RKT), Mesos
	Provisioning	Ansible, Puppet, Chef, Salt, Otto, Vagrant
	Machine Management	AWS, Azure, GCE, vSphere, OpenStack, VirtualBox, Fusion

Container Orchestrators

- **Docker Swarm**: Docker Swarm is a container orchestrator provided by Docker, Inc. It is part of Docker Engine.
- **Kubernetes**: Kubernetes was started by Google, but now, it is a part of the Cloud Native Computing Foundation project.
- **Mesos Marathon**: Marathon is one of the frameworks to run containers at scale on Apache Mesos.
- **Amazon ECS**: Amazon EC2 Container Service (ECS) is a hosted service provided by AWS to run Docker containers at scale on its infrastructure.
- **Hashicorp Nomad**: Nomad is the container orchestrator provided by HashiCorp.

ENTER OUR FRIENDS GOOGLE!

- 2 Billion containers started per week
- 15 + years of container orchestration
- Creators of Borg & Omega
- Learnt many, many lessons
- Distributed systems DNA
  - cgroups
  - artificial intelligence
  - mapreduce
  - bigtable

"Everything at Google runs as a container … everything!"

Joe Beda, Kubernetes founder

Kubernetes has Unprecedented Community Stats

- Most active GitHub project out of 3.6M
- 7K professionals list Kubernetes

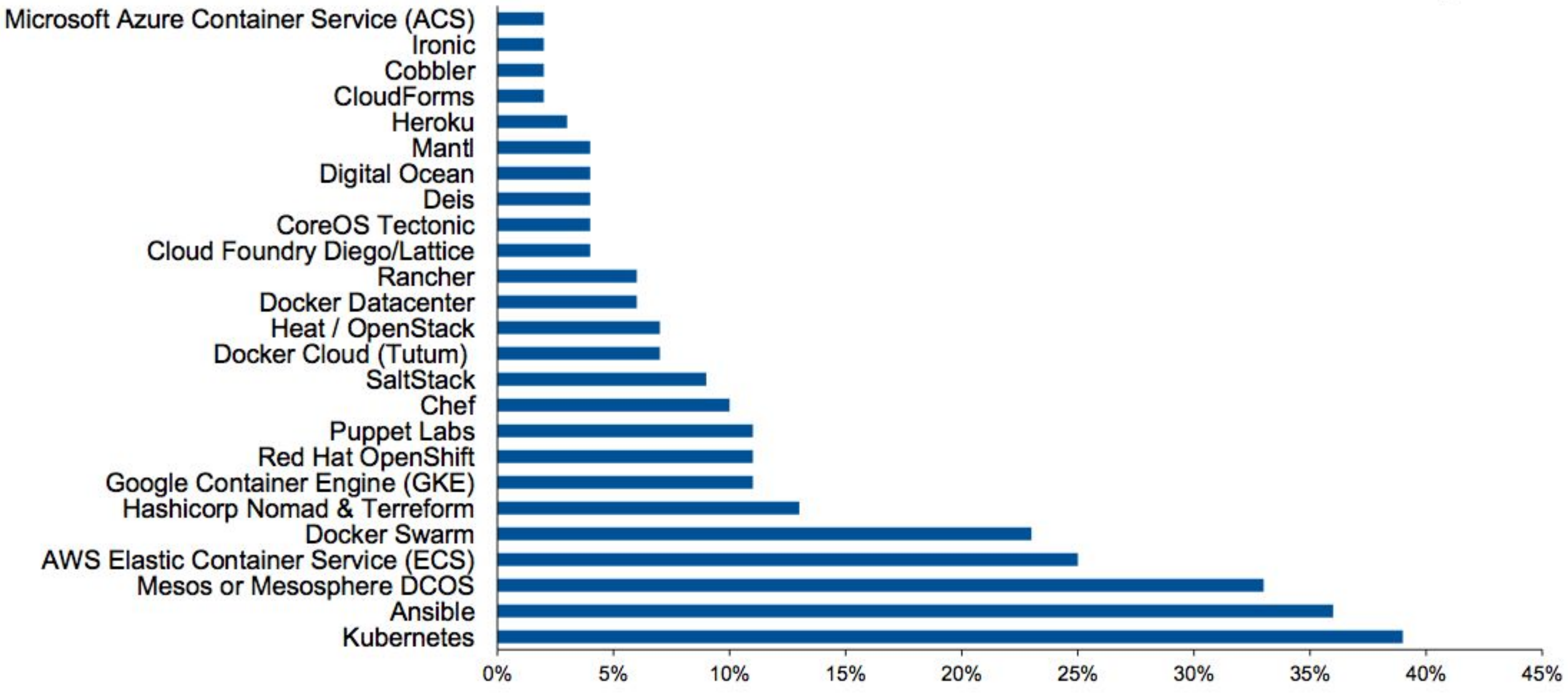
on their LinkedIn profiles

- Largest number of vendors

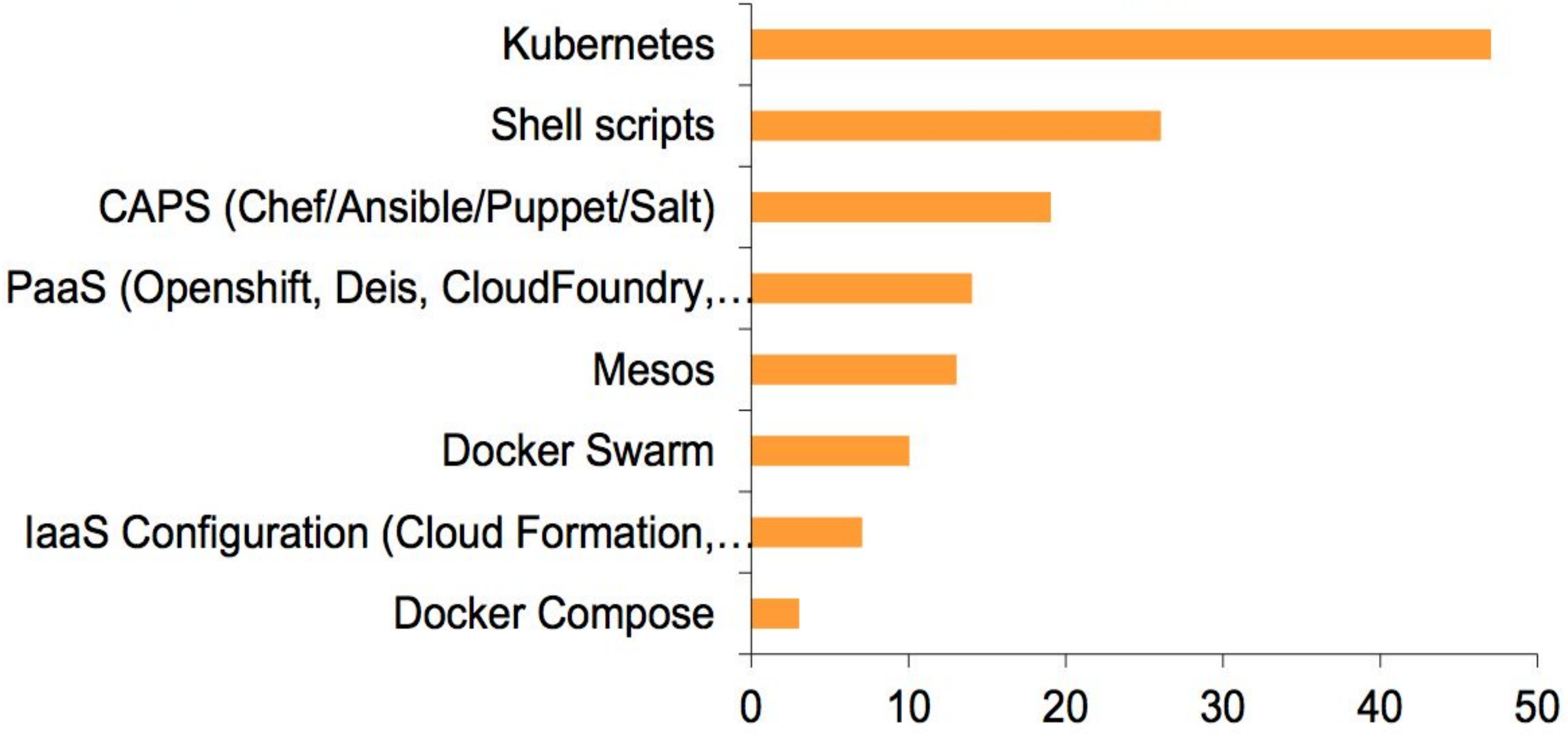
and providers, hedges against vendor lock-in

GITHUB		
36,000+ COMMITS	160+ RELEASES	900+ CONTRIBUTORS
Top 100 FORKED GITHUB PROJECT	Top 2 STARRED GO PROJECT	Top 0.01% STARRED GITHUB PROJECT

Orchestration Technology: Based on Expected Use



"I Manage Containers With…"



Source: Gartner

Kubernetes

Greek for *“Helmsman”*; also the root of the words *“governor”* and *“cybernetic”*

- Production tested solutions at scale (by google) for a decade and a half
- Supported by large community and big Companies like Google, Red Hat, IBM, CoreOS, Intel
- Support for a wide variety of Cloud providers (Azure, Rackspace, vSphere, AWS) and bare-metal environments,
- Supports multiple container runtimes: Docker, RKT (CRI-O soon),
- 100% Open source, written in Go,
- 1st active orchestration project,
- 16000 commits over the last year

Installation Options for a Prod. k8s Cluster

1. Manual installation - “The hard way”
   - The user downloads all components on each node, creates config files, etc.
   - Easy to get things wrong
2. Configuration management (ansible, chef, puppet, etc.)
   - Lots of assumptions made
   - Some only work with specific cloud providers
   - Some expect internet connectivity
3. “Easy Button”

Installation Options

OPTION	FOCUS	STRENGTH	WEAKNESSES
Public Cloud	Managed cluster	• Easy to use • Includes ancillary features	Doesn’t work in data center
KubeSpray	Kubernetes inside corporate firewall or public cloud	• Easy to use • Community OSS	• Their complexity makes it difficult to debug • Work with only a handful of providers
KOPS	AWS deployments		
Kubeadm	Deployments outside firewall	• Easy-ish to use • Community OSS	Takes decent amount of work for production
Vendor Distros	Deployments managed ops	Robust installation option with vendor support	Single vendor “OSS” (propensity for lock in)

HEALTHY COMMUNITY

#minikube

630+ MEMBERS

3000+ STARS

70+ contributors

>4500 7-DAY active users

SUPPORT FOR

OSX

WINDOWS

LINUX

MAINTAINERS

Dan Lorenc
Google

Aaron Prindle
Google

Matt Rickard
Google

Jimmi Dyson
Red Hat

Lucas Käldström

localkube Originally Donated by Redspread CEO Mackenzie Burnett

Resource allocation

Participants	Local VM	GCP VMs		GKE

Create a cluster with kubeadm

1. Provision a Linux machine with Ubuntu, Debian, RHEL, CentOS or Fedora
2. Install kubeadm:

```
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-key add -
echo “deb http://apt.kubernetes.io/ kubernetes-xenial main” > /etc/apt/sources.list.d/kubernetes.list
apt-get update && apt-get install -y kubeadm docker.io
```

3. Make kubeadm set up a master node for you (Link):

```
kubeadm init
```

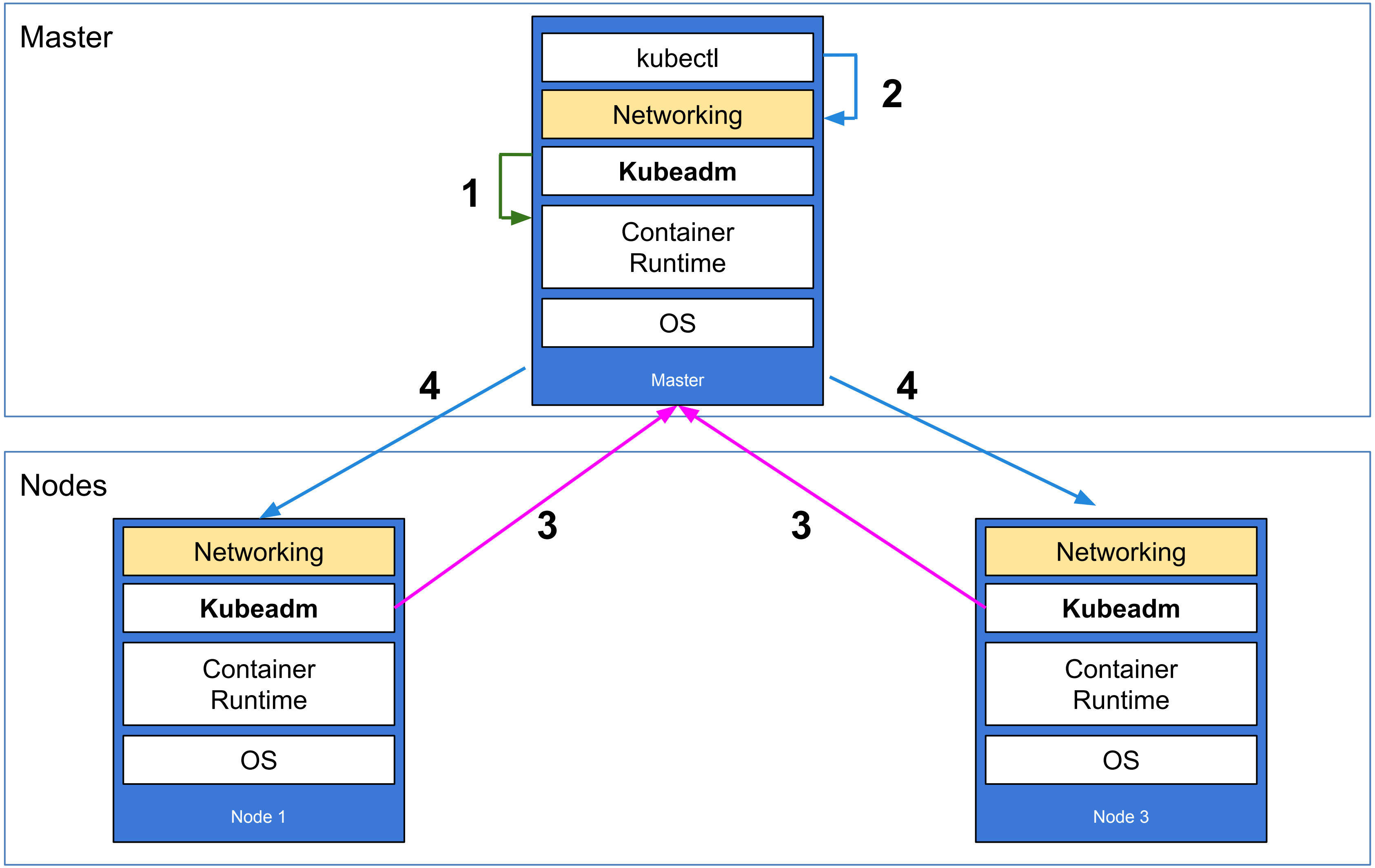
4. Install a Pod Network solution from a third-party provider (Link):

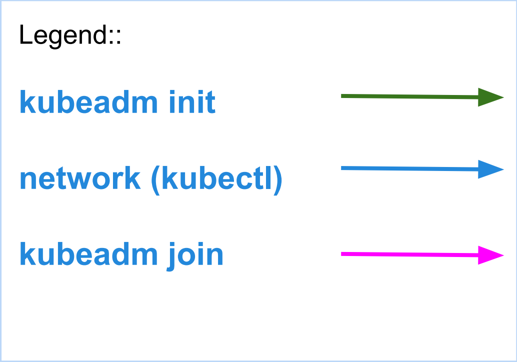
```
kubectl apply -f https://git.io/weave-kube-1.6
```

5. Repeat step 1 & 2 on another node and join the cluster:

```
kubeadm join --token <token> <master-ip>:6443
```

Create a cluster with kubeadm



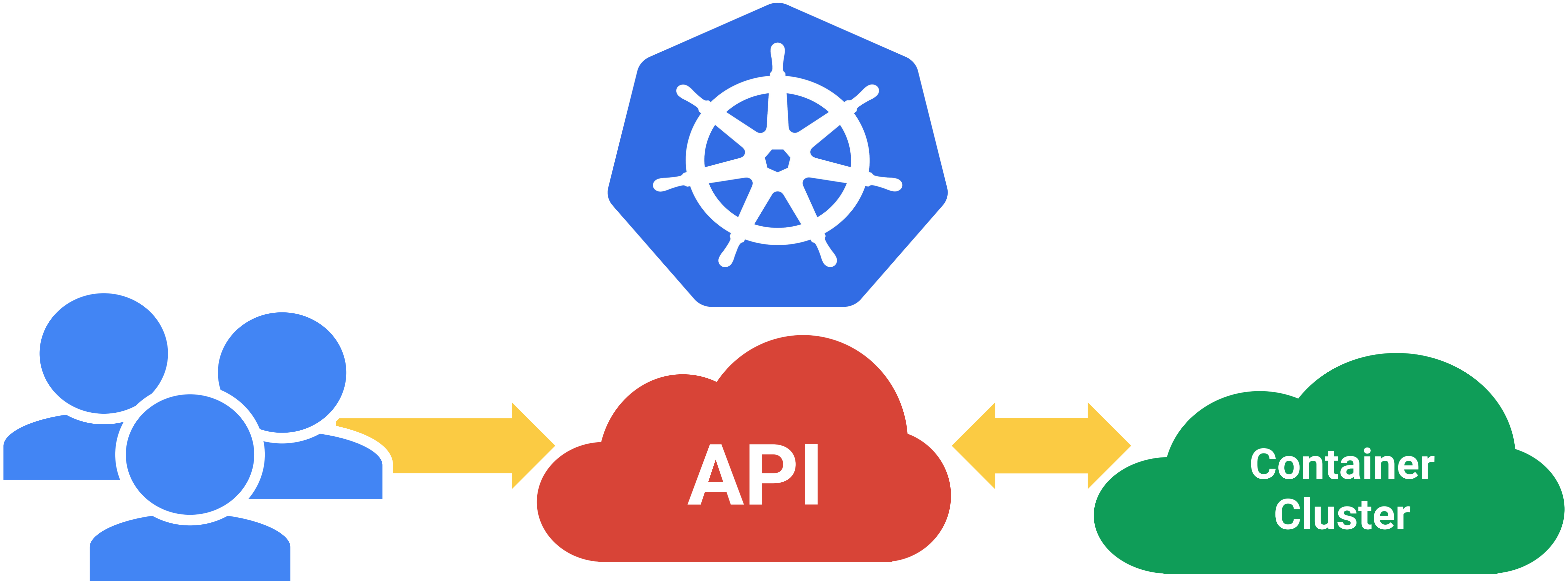


Network providers: Criteria of comparison

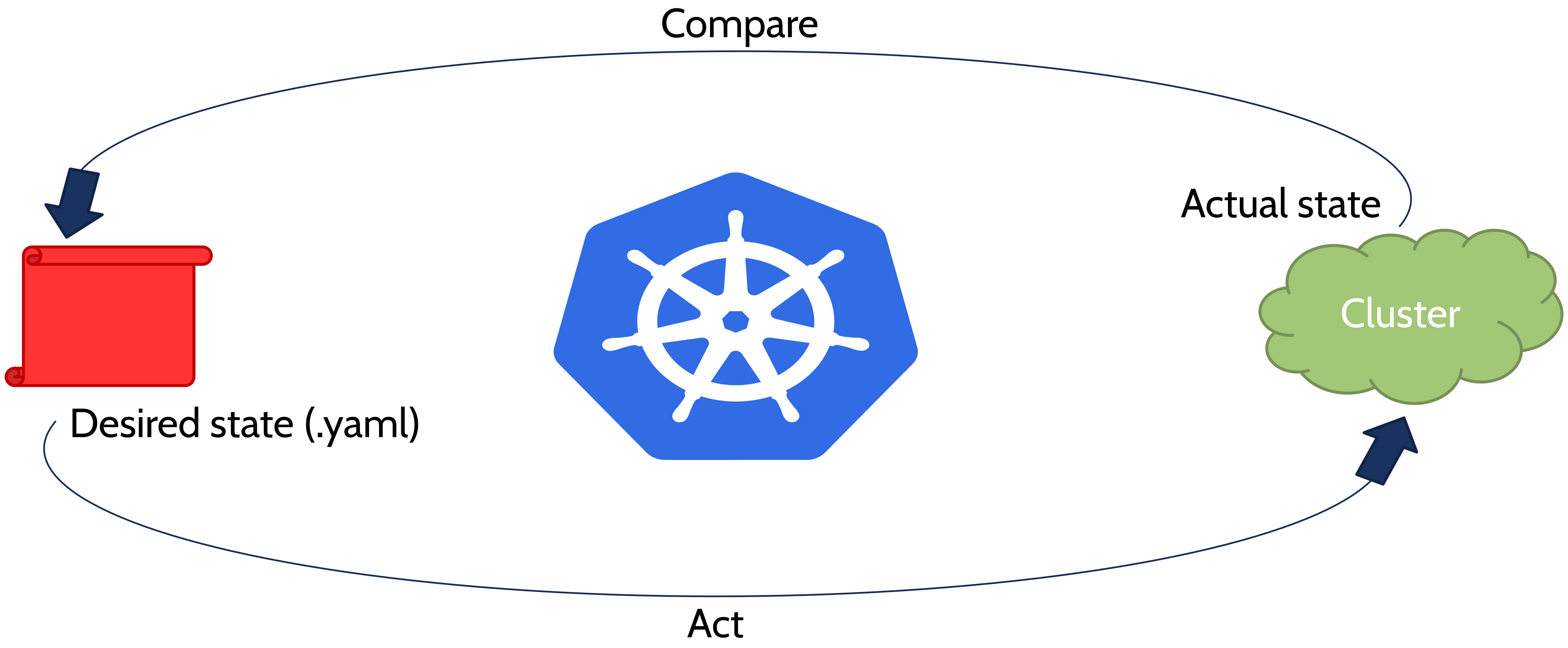
Provider	Network Model	Route Distribution	Network Policies	Mesh	External Datastore	Encryption	Ingress/Egress Policies
Canal	Encapsulated (VXLAN)	No	Yes	No	K8S API	No	Yes
Flannel	Encapsulated (VXLAN)	No	No	No	K8S API	No	No
Calico	Unencapsulated	Yes	Yes	Yes	Etcd	Yes	Yes
Weave	Encapsulated	Yes	Yes	Yes	No	Yes	Yes

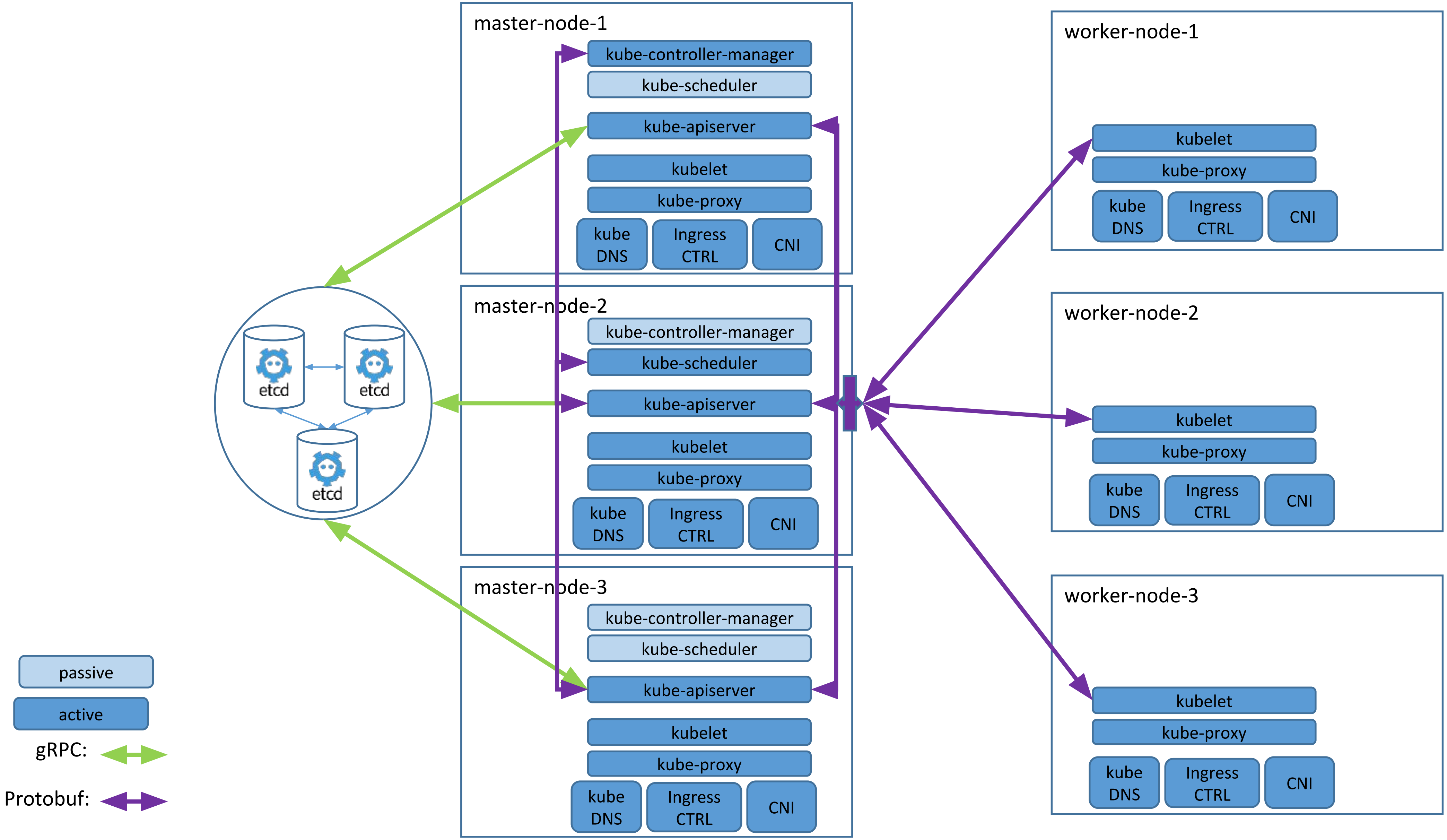
Route Distribution: *An exterior gateway protocol designed to exchange routing and reachability information on the Internet. BGP can assist with pod-to-pod networking between clusters. This feature is a must on unencapsulated CNI network providers, and it is typically done by BGP. If you plan to build clusters split across network segments, route distribution is a feature that's nice-to-have.*

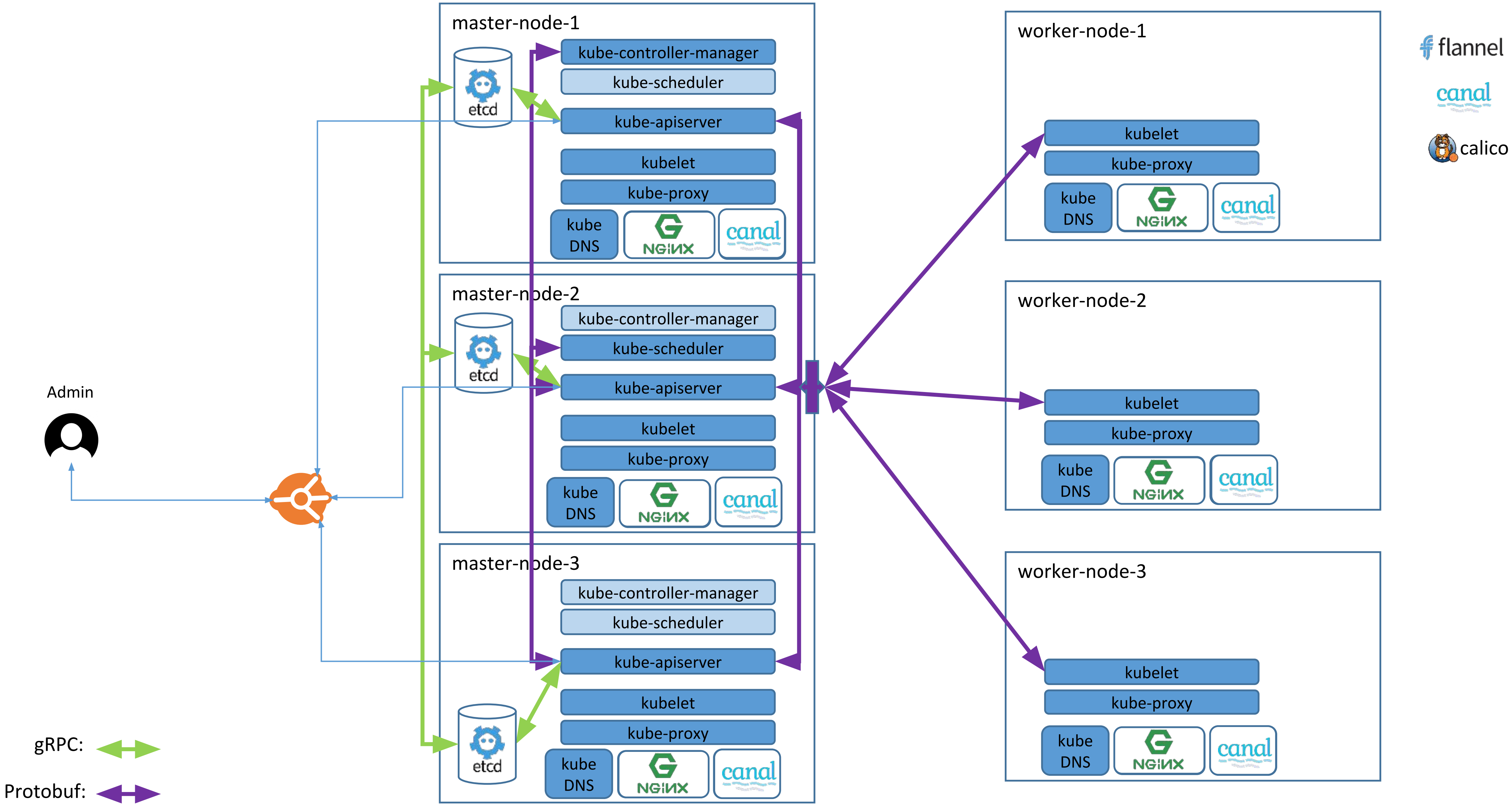
Kubernetes is the Linux for distributed systems



The reconciliation loop

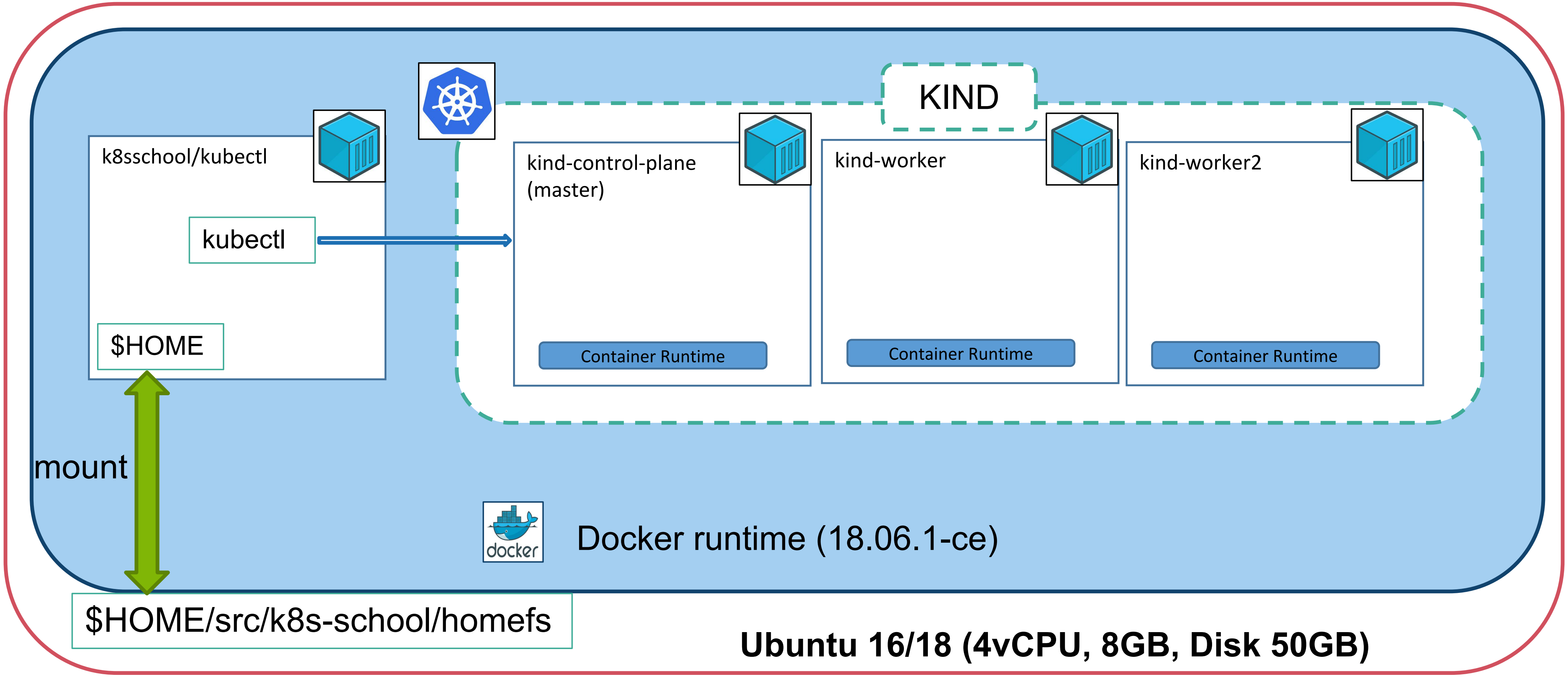



master-node-1
kube-controller-manager
kube-scheduler
kube-apiserver
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
master-node-2
kube-controller-manager
kube-scheduler
kube-apiserver
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
master-node-3
kube-controller-manager
kube-scheduler
kube-apiserver
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
etcd
etcd
etcd
worker-node-1
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
worker-node-2
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
worker-node-3
kubelet
kube-proxy
kube DNS
Ingress CTRL
CNI
passive
active
gRPC:
Protobuf:

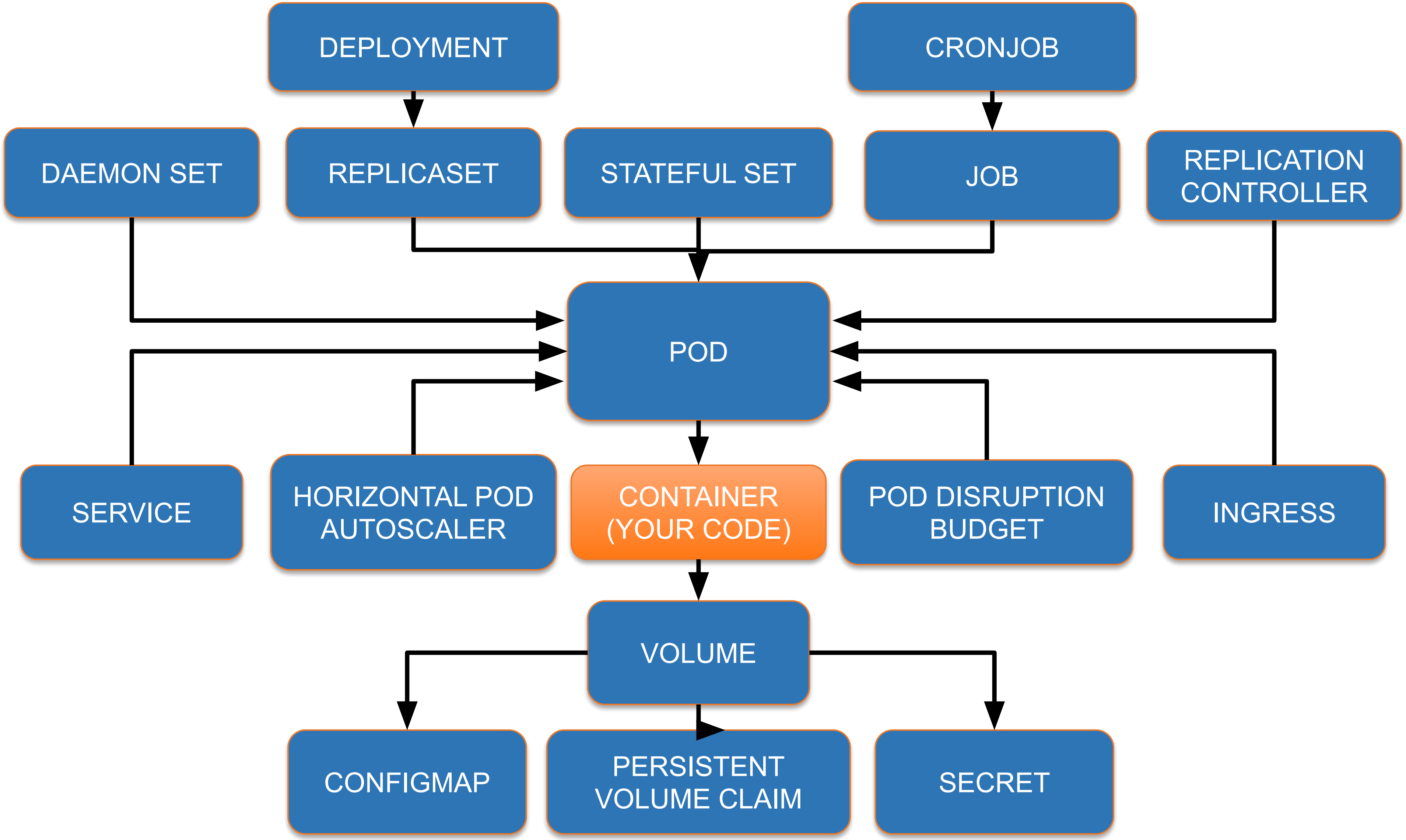


master-node-1
kube-controller-manager
kube-scheduler
etcd
kube-apiserver
kubelet
kube-proxy
kube DNS
NGINX
canal
master-node-2
kube-controller-manager
kube-scheduler
etcd
kube-apiserver
kubelet
kube-proxy
kube DNS
NGINX
canal
master-node-3
kube-controller-manager
kube-scheduler
kube-apiserver
kubelet
kube-proxy
etcd
kube DNS
NGINX
canal
worker-node-1
kubelet
kube-proxy
kube DNS
NGINX
canal
worker-node-2
kubelet
kube-proxy
kube DNS
NGINX
canal
worker-node-3
kubelet
kube-proxy
kube DNS
NGINX
canal
flannel
canal
calico
Admin
gRPC:
Protobuf:

Platform Setup with KIND



https://github.com/kubernetes-sigs/kind
https://kind.sigs.k8s.io/


DEPLOYMENT
CRONJOB
DAEMON SET
REPLICASET
STATEFUL SET
JOB
REPLICATION CONTROLLER
POD
SERVICE
HORIZONTAL POD AUTOSCALER
CONTAINER (YOUR CODE)
POD DISRUPTION BUDGET
INGRESS
VOLUME
CONFIGMAP
PERSISTENT VOLUME CLAIM
SECRET